BEZPIECZEŃSTWO APLIKACJI INTERNETOWYCH I MOBILNYCH

Podstawy inżynierii wstecznej aplikacji na system Android



Autorzy: Amelia Szymańska, Mikołaj Dorosz

INŻYNIERIA WSTECZNA

Czym jest?

01
10

Analiza aplikacji bez dostępu do oryginalnego kodu źródłowego.



Wydobycie szczegółów projektu, algorytmów oraz struktur danych.



Ograniczenie przez przepisy prawne, które różnią się w zależności od kraju.

INŻYNIERIA WSTECZNA

Do czego służy?

- Analiza bezpieczeństwa i wykrywanie podatności.
- Modernizacja przestarzałych systemów.
- Poprawa interoperacyjności.
- Naprawa błędów i optymalizacja.
- Korzystanie z istniejących rozwiązań.



Jak przebiega?

01 **Plan działań:** ustalenie celu, legalności i granic.

02 **APK i środowisko:** pozyskanie pliku APK i przygotowanie środowiska do jego analizy.

03 **Analiza statyczna:** sprawdzenie zawartości pliku instalacyjnego aplikacji APK.

04 **Analiza dynamiczna:** sprawdzenie działania aplikacji w czasie rzeczywistym.

05 **Dokumentacja wyników:** zebranie i wykorzystanie uzyskanych informacji.

Budowa i działanie

Kod źródłowy

Java / Kotlin

C / C++ Native

→

Kompilacja i formaty

Bytecode DEX (Dalvik Executable)

Biblioteki natywne .so (JNI/NDK)

→

Środowisko Android

Środowisko uruchomieniowe ART (AOT / JIT)

4 komponenty: Activity, Service, Receiver, Provider

ANDROID PACKAGE KIT

Czym jest i co zawiera?

APK to pakiet instalacyjny aplikacji, w postaci archiwum ZIP, zawierający wykonywalny kod aplikacji, zasoby, metadane i natywne biblioteki.

AndroidManifest.xml
Metadane, komponenty, uprawnienia.

classes.dex
Skompilowany kod Dalvik/ART reprezentujący kod Java/Kotlin.

res/ i resources.arsc
Obrazy, pliki audio, animacje, style, szablony XML.

assets/ i lib/
Pliki aplikacji i biblioteki natywne.

DALVIK EXECUTABLE (DEX)

Jak go odczytać?

Dekompilacja do Javy/Kotlin

Odtwarza czytelny dla człowieka kod źródłowy z pliku wykonywalnego aplikacji.

- Szybki przegląd logiki biznesowej i architektury programu
- Wynik jest tylko przybliżeniem, zawiera nieścisłości względem oryginału
- Obfuskacja - skraca nazwy zmiennych
- Wymagana korekcja w celu ponownej kompilacji

Deasemblacja do Smali

Tłumaczy kod Dalvik/ART na niskopoziomowy język Asembler.

- Wymaga znajomości operacji na rejestrach
- Wiernie odwzorowuje oryginalny DEX.
- Niewrażliwy na obfuskację
- Pozwala na modyfikację instrukcji i ponowne złożenie APK

ANALIZA STATYCZNA

Przedmioty



Manifest: AndroidManifest.xml — nadmiarowe uprawnienia (SMS, lokalizacja) i komponenty dostępne dla innych aplikacji (android:exported="true").



Kod i zasoby: adresy serwerów i endpointy API, hasła, tokeny i klucze, sposób przechowywania danych, algorytmy kryptograficzne, niebezpieczne funkcje



Konfiguracje i zależności: network_security_config.xml, nieszyfrowany HTTP i cleartextTrafficPermitted; podatne wersje bibliotek firm trzecich.



Weryfikacja: znalezisko to hipoteza, nie dowód. Martwy kod lub zabezpieczenia dynamiczne mogą uniemożliwić jej wykorzystanie.

ANALIZA STATYCZNA

Narzędzia



JADX

Dekompilator DEX/APK.
Wyszukiwanie klas, metod i tekstu.



Apktool

Deasembler DEX/APK.
Odczytywanie kodu i zasobów.



SISK

Online APK Analyzer.



Android Studio

IDE Google i APK Analyzer.

ANALIZA DYNAMICZNA

Czym jest?

Polega na badaniu działającego oprogramowania w czasie rzeczywistym



Pozwala obserwować: zachowanie aplikacji w czasie rzeczywistym, komunikację z serwerem, logi systemowe, wywołania funkcji i reakcję na dane użytkownika.

Techniki

1. **Monitorowanie ruchu sieciowego**

   Analiza komunikacji HTTP/HTTPS pomiędzy aplikacją a serwerem, np. żądań i odpowiedzi API.

1. **Instrumentacja i analiza funkcji**

   Obserwowanie wywołań metod, ich parametrów i wyników, a w testach także zmienianie zachowania funkcji.

1. **Monitorowanie logów systemowych**

   Analiza komunikatów generowanych przez aplikację podczas działania, np. błędów lub przypadkowo ujawnionych danych.







Narzędzia analizy dynamicznej

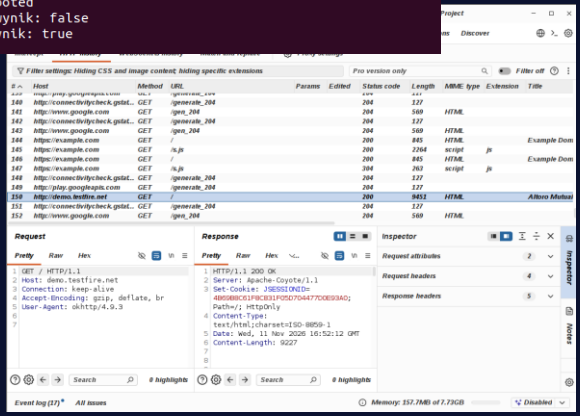
Frida

Obserwacja wywołań metod i testowa zmiana działania aplikacji.

Burp Suite

Przechwytywanie i analiza komunikacji HTTP/HTTPS między aplikacją a serwerem.

```
amellaszyamnska@bertold:~/AndroidLabs$ nano ~/AndroidLabs/hook.js
amellaszyamnska@bertold:~/AndroidLabs$ cd ~/AndroidLabs
frida -U -n Gadget -l hook.js
    ____
   / _  |   Frida 17.26.2 - A world-class dynamic instrumentation toolkit
  | (_| |
   > _  |   Commands:
  /_/ |_|       help      -> Displays the help system
  . . . .       object?   -> Display information about 'object'
  . . . .       exit/quit -> Exit
  . . . .
  . . . .   Prefer a GUI? Luma is the official Frida app, with a live REPL,
  . . . .   persistent sessions & collaboration. https://luma.frida.re/
  . . . .
  . . . .   Connected to SM A536B (id=RZCT90ZQ3XM)
Attaching...
[Frida] Hook aktywny
[SM A536B::Gadget ]-> [Frida] Metoda: isRooted
[Frida] Oryginalny wynik: false
[Frida] Zmieniony wynik: true
[Frida] Metoda: isRooted
[Frida] Oryginalny wynik: false
[Frida] Zmieniony wynik: true
```



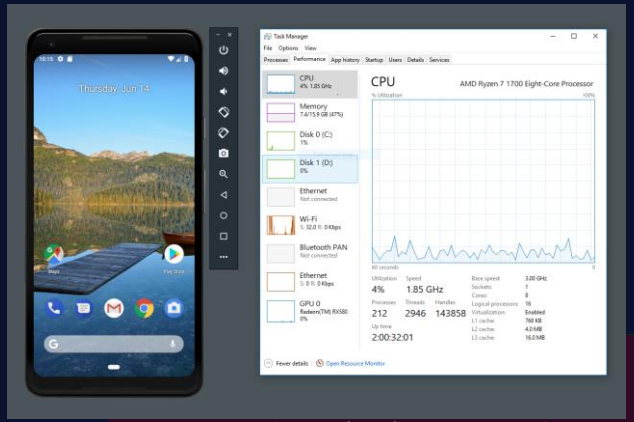
Narzędzia analizy dynamicznej

Android Emulator · środowisko

Środowisko testowe do uruchamiania i badania aplikacji Android.

Logcat

Podgląd logów generowanych przez aplikację i system Android.



```
                                                                    isPredictionOn
                                                                    isPredictionOn
11-11 17:09:21.910 I/HBD      ( 3715): c isTextPasswordInputType    isPredictionOn
 = false
11-11 17:09:21.920 I/HBD      ( 3715): c isTextPasswordInputType    isPredictionOn
 = false
11-11 17:09:21.920 I/HBD      ( 3715): c isTextPasswordInputType    isPredictionOn
 = false
11-11 17:09:21.920 I/HBD      ( 3715): c isTextPasswordInputType    isPredictionOn
 = false
11-11 17:09:21.921 I/HBD      ( 3715): c isTextPasswordInputType    isPredictionOn
 = false
11-11 17:09:50.863 D/WifiProfileShare.McfCast( 1334): -ME--- updatePasswordDate
 pwdData size:1
11-11 17:10:20.929 D/WifiProfileShare.McfCast( 1334): -ME--- updatePasswordDate
 pwdData size:1
11-11 17:10:25.540 I/Info:    (20954): Username: amelia and Password: test12eq a
re verified
11-11 17:10:25.540 I/System.out(20954): Username: amelia and Password: test12eq
 are verified
```

ZAGROŻENIA

Co może ujawnić inżynieria wsteczna?

01 Zaszyte sekrety

Hasła, poufne klucze API i tokeny zapisane w kodzie aplikacji.

02 Niezabezpieczone dane

Dane wrażliwe przechowywane w plikach, bazach danych lub logach.

03 Słaba komunikacja

Błędy konfiguracji TLS i nieprawidłowa weryfikacja certyfikatów.

04 Błędy autoryzacji

Decyzje o dostępie podejmowane wyłącznie po stronie aplikacji.

ZAGROŻENIA

Modyfikacja logiki aplikacji

ORYGINALNA APLIKACJA

```
if (isPremiumUser) {
    showContent();
}
```

Dostęp zależy od wyniku sprawdzania.

ZMODYFIKOWANA APLIKACJA

```
isPremiumUser = true;
showContent();
```

Lokalny warunek zostaje ominięty.

! **Krytyczna autoryzacja nie może zależeć wyłącznie od kodu klienta.**

ZABEZPIECZENIA

Jak utrudnić inżynierie wsteczną?

01	**Obfuskacja kodu**	Utrudnianie zrozumienia klas, metod i logiki programu.
02	**Wykrywanie debugowania**	Utrudnianie śledzenia działania aplikacji podczas testów.
03	**Kontrola integralności**	Wykrywanie nieoczekiwanych zmian w aplikacji i jej środowisku.
04	**Weryfikacja po stronie serwera**	Niezależne sprawdzanie tożsamości i uprawnień użytkownika.

Mechanizmy ochronne utrudniają analizę — nie zapewniają całkowitej odporności na modyfikacje.

ZABEZPIECZENIA

Ochrona danych i komunikacji

01	**Bezpieczne zarządzanie sekretami**	Nieumieszczanie poufnych kluczy serwerowych w APK.
02	**Bezpieczne przechowywanie**	Android Keystore i odpowiednia ochrona lokalnych danych.
03	**Ochrona transmisji**	TLS oraz prawidłowa weryfikacja certyfikatów serwera.
04	**Ograniczenie danych w logach**	Niezapisywanie haseł, tokenów i innych poufnych informacji.

Najważniejsza zasada: nie polegaj wyłącznie na ukryciu kodu aplikacji.

PODSUMOWANIE

Najważniejsze wnioski

01

Gotowy APK można analizować

Brak kodu źródłowego nie oznacza braku możliwości poznania działania aplikacji.

02

Dwie metody się uzupełniają

Analiza statyczna bada kod i pliki, a dynamiczna obserwuje zachowanie programu.

03

Kod klienta może zostać zmieniony

Lokalne warunki i sekrety nie są wystarczającą ochroną krytycznych funkcji.

04

Bezpieczeństwo wymaga wielu warstw

Ochrona danych, kontrola po stronie serwera oraz utrudnianie modyfikacji.

Najważniejsze: aplikację można badać i zmieniać — bezpieczeństwo nie może opierać się na ukryciu kodu.

PODSUMOWANIE

Źródła

- https://www.aspiresys.com/blog/digital-software-engineering/agile-software-solutions/the-essential-concepts-behind-software-reverse-engineering/
- https://developer.android.com/guide/components/fundamentals
- https://developer.android.com/studio/debug/apk-analyzer
- https://medium.com/@povzayd/reverse-engineering-an-android-apk-theory-architecture-focused-14970a961619
- https://docs.hex-rays.com/core/decompiler/overview/introduction-to-decompilation-vs-disassembly
- https://medium.com/@niloycd/beginners-guide-to-reverse-engineering-with-android-apps-cc3fb1a962ad
- https://cellebrite.com/en/blog/intro-to-android-app-reverse-engineering/
- https://portswigger.net/burp

Dziękujemy za uwagę!